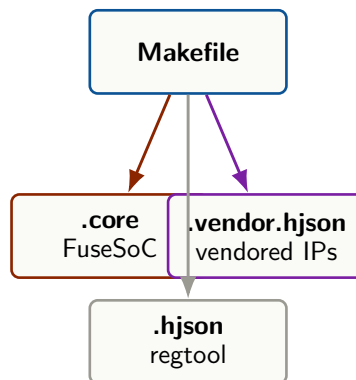


A QUICK REFERENCE MANUAL

FuseSoC, Vendorsing, Makefiles & Regtool

Reading (and Writing) the Lab Build Files and Register Descriptions



Luigi Giuffrida

Department of Electronics and Telecommunications
Politecnico di Torino

Draft — September 17, 2026

This booklet is a companion reference, not a build-systems course. It covers exactly enough of FuseSoC, vendoring, `make` and the `register_interface` regtool to read the `.core` files, Makefiles and `.hjson` register descriptions that ship with the lab folders, and to write new ones of your own. For anything beyond this scope, the official FuseSoC documentation at fusesoc.readthedocs.io, GNU `make`'s manual, and the OpenTitan regtool documentation are the authoritative sources.

Contents

1	FuseSoC and the .core File	1
1.1	What FuseSoC is	1
1.2	Anatomy of a .core file	1
1.3	Targets: the same sources, run differently	3
1.4	Running it	3
1.5	Where the build goes	4
1.6	Writing your own .core file	4
1.7	Checklist	4
2	Vendoring: Pulling In Other People's Cores	7
2.1	What “vendoring” means here	7
2.2	Anatomy of a .vendor.hjson	7
2.3	What make vendor actually does	8
2.4	Two vendoring patterns in these labs	9
2.5	Checklist	9
3	Makefiles: Where the Commands Live	11
3.1	Why bother with a Makefile	11
3.2	The vocabulary you need, and no more	11
3.3	Pattern 1: a thin wrapper around FuseSoC	12
3.4	Pattern 2: a forwarding Makefile	12
3.5	Reading a Makefile you have not seen before	13
3.6	Adding a target of your own	13
3.7	How it all fits together: one <code>make sim run</code>	13
3.8	Checklist	14
4	The Register Interface and Regtool	15
4.1	Two buses, one job each	15
4.2	Anatomy of the .hjson register description	15
4.3	What make regs actually runs	16
4.4	Reading the generated types	17
4.5	Wiring it into your design	17
4.6	The C side	18

4.7 Checklist	18
A One-Page Quick Reference	19
A.1 Commands	19
A.2 .core file fields	19
A.3 .vendor.hjson fields	20
A.4 .hjson register description fields	20
A.5 Writing a new .core file, in five steps	20
A.6 The three traps to remember	21

FuseSoC and the .core File

What FuseSoC actually does, how a `.core` file describes a design, and how to read one of the lab's `.core` files end to end. Ends with a checklist for writing your own.

1.1 What FuseSoC is

FuseSoC is a package manager and build tool for hardware sources. It solves one problem: a design is rarely a pile of files handed straight to a simulator. It depends on other designs (IPs), it needs a different file list for simulation than for synthesis, and each tool (Verilator, QuestaSim, Design Compiler, ...) wants its options passed differently. FuseSoC lets you describe all of that once, declaratively, in a `.core` file, and then resolves it: given a target (“simulate”, “lint”, “synthesise”), it works out which files are needed, in which order, pulls in whatever those files themselves depend on, and calls the right tool with the right flags.

Every design and every dependency, however small, is a **core**: a `.core` file plus the sources it points at. A lab accelerator is a core. `common_cells` is a core. X-HEEP itself is a core. FuseSoC does not care how big a core is, only that it has a name and a `.core` file describing it.

A `.core` file does not contain RTL. It only says which files exist, what they depend on, and how to run them under each target. Read it as a table of contents plus a recipe, never as an implementation.

1.2 Anatomy of a .core file

Every `.core` file in these labs has the same four kinds of section, in the same order: a header, a name and description, one or more **filesets**, and one or more **targets**. Here is the whole of `cordic_accel.core` from Lab 1, the smallest real example in the series.

```

CABI=2:
name: isa:lab1:cordic_accel:0.1.0
description: CORDIC accelerator with an OBI data port and a reggen CSR interface

filesets:
  rtl:
    files:
      - rtl/cordic_pkg.sv
      - rtl/cordic_accel.sv
    file_type: systemVerilogSource
  depend:
    - pulp-platform.org::obi
    - pulp-platform.org::register_interface
    - pulp-platform.org::common_cells

```

```

tb:
  files:
    - tb/tb_cordic_accel.sv
  file_type: systemVerilogSource
  depend:
    - pulp-platform.org::obi_test
    - pulp-platform.org::reg_test

targets:
  default: &default
  filesets: [rtl]

sim:
  <<: *default
  default_tool: verilator
  filesets: [rtl, tb]
  toplevel: tb_cordic_accel
  tools:
    verilator:
      mode: binary
      verilator_options: [--timing, --trace-fst]

lint:
  <<: *default
  default_tool: verilator
  toplevel: cordic_accel
  tools:
    verilator:
      mode: lint-only

```

Listing 1.1. cordic_accel.core – lab1/cordic_accel.core (trimmed)

1.2.1 The header and the name

CAP1=2: is not a comment: it is the first line FuseSoC’s parser looks for, and it says “this file uses Core API version 2”. Every .core file in these labs starts with it.

The **name** is a colon-separated identifier, conventionally <vendor>:<library>:<core>:<version>: isa:lab1:cordic_accel:0.1.0 reads as “the ISA lab group’s lab1 library, core cordic_accel, version 0.1.0”. This is exactly what another .core file names in its own **depend** list to pull this core in; Chapter 2 shows the vendored IPs’ names the same way (pulp-platform.org::obi and similar).

1.2.2 Filesets

A **fileset** is a named group of files that belong together for one purpose: rtl for the synthesisable design, tb for the testbench, and so on – the names themselves are not special to FuseSoC, only convention. Each fileset lists:

- **files**: paths relative to the .core file itself, in compile order where the tool cares (packages before the modules that use them).
- **file_type**: how the tool should treat them – systemVerilogSource, vhd1Source, cSource, and so on.
- **depend**: other cores’ names. FuseSoC resolves these recursively and adds their own filesets to the build, so you never list a dependency’s files by hand.

△ Trap: A fileset is not a target

Declaring `rtl:` and `tb:` does not build anything by itself. Nothing runs until a *target* lists which filesets it needs. It is entirely normal for a `.core` file to declare a fileset that only one target ever uses (Listing 1.1’s hypothetical synthesis-only technology cells, for instance).

1.3 Targets: the same sources, run differently

A **target** is a named recipe: which filesets to pull in, which tool to run, which module is the top of the design, and any tool-specific options. The real `cordic_accel.core` (Chapter 2 onwards use the un-trimmed file) has five: `default`, `sim` (Verilator), `sim_questa` (QuestaSim, same testbench), `synth` (Design Compiler) and `lint` (Verilator, lint-only). They all draw from the same two or three filesets; only the combination and the tool options change.

The `&default` / `<: *default` trick

`&default` and `<: *default` are plain YAML anchors and merge keys, not FuseSoC syntax: `&default` names the `default:` block, and `<: *default` copies its keys into another block before overriding some of them. It exists purely so you do not repeat filesets: `[rtl]` in every target.

`toplevel` names the module the tool should elaborate from; it can differ between targets in the same file (Listing 1.1’s `sim` target elaborates the testbench, `lint` elaborates the accelerator itself, since there is no testbench to lint). The `tools` block holds one sub-block per tool named in `default_tool`, with whatever options that specific tool understands – `verilator_options` for Verilator, `vlog_options` for QuestaSim’s compiler, and so on. FuseSoC does not validate these against the tool; a typo here fails at the tool invocation, not at parse time.

1.4 Running it

```
fusesoc --cores-root . --cores-root vendor run --target sim isa:lab1:cordic_accel
```

Reading left to right: `-cores-root <dir>` tells FuseSoC where to look for `.core` files, and it needs both the lab directory itself (for `cordic_accel.core`) and `vendor/` (for the `.core` files describing the vendored IPs it depends on) – omit either and resolution fails with “core not found”. `run -target sim` picks the target block by name. The last argument is the core’s `name`. FuseSoC then resolves the dependency tree, writes a file list into a `build/<core>/` directory, and invokes the target’s `default_tool` on it.

Recipe: Let the Makefile remember the flags

Nobody types the `-cores-root` flags by hand every time; that is exactly what the lab Makefiles are for. Chapter 3 shows the wrapper.

1.5 Where the build goes

Every run writes into `build/<core-name>/<target>-<tool>/`, with colons in the core’s `name` turned into underscores – `isa:lab1:cordic_accel:0.1.0` and target `sim` become `build/isa_lab1_cordic_accel_0.1.0_sim`. Inside that folder: the file list FuseSoC derived (`*.eda.yml`), a generated `Makefile` that actually drives the tool, and every intermediate and output file the tool produces – for Verilator, that includes the compiled simulation binary and, once you run it, the waveform dump. None of it is meant to be edited or committed; delete the whole `build/` tree whenever a run looks confused and re-run `fusesoc` to start clean.

```
fusesoc --cores-root . --cores-root vendor core-info isa:lab1:cordic_accel
fusesoc --cores-root . --cores-root vendor list-cores
```

`core-info` prints the resolved `.core` file – filesets, targets and all – exactly as FuseSoC parsed it, which is the fastest way to catch a YAML typo or a missing `depend` before running anything. `list-cores` lists every core FuseSoC can currently see across the given `-cores-root` directories; if your own core is missing from it, the directory holding its `.core` file was never passed in.

△ Trap: `-Wno-fatal` is not “skip linting”

Listing 1.1’s real counterpart adds `-Wno-fatal` to the `sim` target’s Verilator options, with a comment worth reading literally: “*The vendored IPs are lint-clean for their own flow, not for ours.*” Verilator still reports every warning; `-Wno-fatal` only stops a warning from *aborting* the build, so warnings in `vendor/` code (built against a different lint configuration upstream) do not block a simulation of your own module. It does not silence output and it is not a licence to ignore warnings in the RTL you wrote yourself – lint runs without it for exactly that reason.

1.6 Writing your own .core file

Copying `cordic_accel.core` and editing it is the right way to start; these are the pieces to change, in order.

1. **Name it.** Pick `<yourvendor>:<lab>:<core>:<version>`. This is the string every dependant, and every `fusesoc` run, will use to refer to it.
2. **List your sources in a fileset.** Compile order matters for SystemVerilog packages: a `_pkg.sv` file goes before anything that imports it.
3. **Declare dependencies by name**, not by copying their files into your fileset. If a dependency is not already vendored, Chapter 2 is the next stop, not this file.
4. **Add a target for the tool you actually run.** Start from the smallest working target (usually `lint`, since it needs no testbench) and add `sim` once the design elaborates cleanly.
5. **Set `toplevel`** to whatever module that target should elaborate – the design for `lint`, the testbench for `sim`.

1.7 Checklist

- A `.core` file describes files and how to run them; it never contains RTL itself.
- A fileset groups files with a type and a dependency list; a target picks filesets, a tool, and a `toplevel`.

- `depend` names another core; FuseSoC resolves it, you do not list its files.
- `fusesoc run` always needs every `-cores-root` that holds a `.core` file the build touches, your own directory included.
- When something does not build, read the target block first: wrong filesets or the wrong toplevel accounts for most first mistakes.

Vendoring: Pulling In Other People's Cores

Where the code behind a `depend` entry in a `.core` file actually comes from: a pinned, reproducible snapshot copied in by a small Python tool, not a git submodule.

2.1 What “vendoring” means here

A `.core` file can declare `depend: pulp-platform.org::obi`, but that only works once the files of that dependency actually exist somewhere FuseSoC can find them. **Vendoring** is how they get there: a script fetches a pinned snapshot of an upstream repository and copies it, as plain files, into a `vendor/` directory inside the lab. No git submodule, no `.gitmodules`, and after the copy, no connection back to the upstream repository's own `.git` history at all.

Vendoring answers “where do these files come from and which exact version are they”. FuseSoC's `depend` answers “which files does this target need”. The two are independent: FuseSoC does not fetch anything, and the vendoring script does not know what a target is.

Why not a submodule?

A submodule keeps the upstream's whole history and its own `.git`, which a student clone does not need and a build tool should not have to walk. A vendored copy is just files: reproducible from the pinned `rev`, easy to diff, and impossible to accidentally git push into.

2.2 Anatomy of a `.vendor.hjson`

Every dependency the lab vendors has its own descriptor, ending in `.vendor.hjson`. Two real ones, at opposite ends of the size scale.

```
// Vendoring descriptor: pulp-platform common_cells.
// Fetched by `make vendor` (python3 util/vendor.py <this file>).
{
  name: "common_cells",
  target_dir: "common_cells",

  upstream: {
    url: "https://github.com/pulp-platform/common_cells.git",
    rev: "v1.38.0",
  },

  exclude_from_upstream: [
    "test",
    "ci",
  ],
}
```

```

    "formal",
    ".github",
  ]
}

```

Listing 2.1. lab1/vendor/pulp_platform/common_cells.vendor.hjson

```

// Vending descriptor for X-HEEP.
// `make vendor` runs: python3 util/vendor.py x-heap.vendor.hjson
{
  name: "x-heap",
  target_dir: "x-heap",

  upstream: {
    url: "https://github.com/esl-epfl/x-heap.git",
    rev: "main",
  },
}

```

Listing 2.2. lab0/x-heap.vendor.hjson (the whole file)

The fields are the same regardless of size:

- **name**: a label for the descriptor itself, not a FuseSoC core name.
- **target_dir**: where the snapshot lands, relative to **vendor/** (or wherever the enclosing Makefile passes it).
- **upstream**: **url** plus **rev** – a tag, branch, or commit SHA. This is the whole reproducibility story: two people running **make vendor** against the same descriptor get byte-identical sources.
- **exclude_from_upstream** (optional): paths to drop from the copy – typically the upstream’s own tests and CI, which the lab does not need and does not want cluttering **vendor/**.

△ **Trap: rev: "main" is a moving target**

Listing 2.2 pins to **main** on purpose while the lab is still being developed: it always vendors the latest X-HEEP. Once a lab is handed to students, **rev** should be pinned to a commit SHA – otherwise two students running **make vendor** on different days can end up with different X-HEEP versions and different bugs.

2.3 What make vendor actually does

```
python3 util/vendor.py vendor/pulp_platform/common_cells.vendor.hjson
```

The script (**util/vendor.py**, itself vendored from OpenTitan and not something you need to read) does three things: fetches the repository at the pinned **rev** into a temporary location, copies everything except **exclude_from_upstream** into **target_dir**, and writes a **<name>.lock.hjson** next to the descriptor recording exactly what was fetched (the resolved commit, the date). The lock file is what you would diff to answer “did this dependency change since last time”.

A lab’s Makefile runs this once per descriptor, over every ***.vendor.hjson** it finds:

```

VENDOR_HJSON := $(wildcard $(VENDOR)/*.vendor.hjson)

.PHONY: vendor
vendor:
    @for f in $(VENDOR_HJSON); do \

```

```
echo "$(PYTHON) util/vendor.py $$f"; \  
$(PYTHON) util/vendor.py $$f || exit 1; \  
done
```

Listing 2.3. lab1/Makefile – the vendor target

2.4 Two vendoring patterns in these labs

Lab 1 vendors several small IPs individually, each with its own `.vendor.hjson` under `vendor/pulp_platform/`, because `cordic_accel.core` depends on each of them by name separately. Lab 0 vendors one thing, X-HEEP itself, as a single large snapshot, because the whole lab is built on top of it rather than depending on named pieces of it. Neither is more correct: vendor at the granularity your `.core` files actually depend on.

2.5 Checklist

- Vendoring copies pinned files in; it is not a submodule and keeps no upstream history.
- `upstream.rev` is the whole reproducibility guarantee – pin it to a SHA or tag before handing a lab out.
- `util/vendor.py <descriptor>.vendor.hjson` does the fetch; `make vendor` just runs it over every descriptor it finds.
- The generated `.lock.hjson` records what was actually fetched; do not hand-edit it.
- A vendored directory is regenerated by `make vendor`, so it is normal for it to be absent from a fresh clone and for `.gitignore` to exclude it.

Makefiles: Where the Commands Live

Why every lab has a Makefile instead of a page of shell commands to copy-paste, the two Makefile shapes used across the labs, and how to add a target to one.

3.1 Why bother with a Makefile

Chapter 1 and Chapter 2 both end in a command line you would not want to retype, with flags you would not want to remember:

```
fusesoc --cores-root . --cores-root vendor run --target sim isa:lab1:cordic_accel \
--run_options="+VECDIR=/home/you/lab1/vectors/"
```

A Makefile turns that into `make sim`. That is the entire point: one file that remembers every incantation, gives each one a short name, and lets `make <target> -dry-run` (or just reading the recipe) show you the real command underneath whenever you need it. Nothing in a lab Makefile is magic; every recipe is a command you could type yourself.

If you cannot see what a Makefile target actually runs, read the recipe line, not the target name. The labs deliberately @echo or leave visible the exact FuseSoC/ Python / make command each target wraps.

3.2 The vocabulary you need, and no more

- A **target** is a name after which a colon introduces its prerequisites, and the indented lines below are its **recipe** (the shell commands `make` runs). Indentation must be a literal tab, not spaces.
- `.PHONY: sim` tells `make` that `sim` is a name for an action, not a file it should check the timestamp of. Every target in these labs is phony.
- `sim: vectors bitexact` means “before running `sim`’s own recipe, make sure `vectors` and `bitexact` have run”.
- Variables come in two common flavours here: `:=` (set once, now) and `?=` (set only if not already set – lets you override on the command line, `make sim NANGLES=128`). `export` makes a variable visible to sub-processes, including another Makefile pulled in with `include`.
- `$(wildcard pattern)` expands to every file matching `pattern` that exists right now – used in Chapter 2 to loop over every `*.vendor.json`.

3.3 Pattern 1: a thin wrapper around FuseSoC

Lab 1's Makefile is the simplest shape: define the flags once, then one target per FuseSoC target, plus a few for things FuseSoC does not do (vendoring, register generation, vector generation).

```
FUSESOC_FLAGS := --cores-root . --cores-root vendor
CORE           := isa:lab1:cordic_accel

.PHONY: sim
sim: vectors bitexact
    $(FUSESOC) $(FUSESOC_FLAGS) run --target sim $(CORE) \
        --run_options="+VECDIR=$(ROOT)/$(VECDIR)/"

.PHONY: lint
lint:
    $(FUSESOC) $(FUSESOC_FLAGS) run --target lint $(CORE)
```

Listing 3.1. lab1/Makefile – the shape, trimmed

Every FuseSoC target from the `.core` file (`sim`, `lint`, `synth`, ...) gets exactly one Makefile target of the same name, and that target's whole job is to fill in `-cores-root` and the core name so you never type them. `sim`'s prerequisites (`vectors bitexact`) show the other half of the job: a Makefile also sequences the non-FuseSoC steps (generate vectors, check them) that have to happen first.

3.4 Pattern 2: a forwarding Makefile

Lab 0 does not wrap FuseSoC calls directly; X-HEEP already ships its own large `external.mk` with targets like `verilator-build` and `app`. Reimplementing those would mean maintaining a second copy of logic that already works, so Lab 0's Makefile instead defines only the handful of targets specific to the lab, then hands everything else to the vendored X-HEEP:

```
export HEED_DIR ?= $(ROOT_DIR)/x-heep
export SOURCE    ?= ../../sw/

.PHONY: help vendor mcu-gen

vendor:
    python3 util/vendor.py x-heep.vendor.hjson

mcu-gen:
    cd x-heep && fusesoc run --target mcu-gen ...

# IMPORTANT: this MUST stay at the very bottom -- its catch-all rule
# swallows every target defined after it.
include x-heep/external.mk
```

Listing 3.2. lab0/Makefile – the shape, trimmed

`include x-heep/external.mk` literally pastes that file's targets in at this point, as if you had typed them here. Because `external.mk` defines a catch-all pattern rule, **anything defined after the include is invisible**: `make` matches the catch-all first and never reaches it. That is why the comment says to keep `include` at the bottom, and why it is the one line in this Makefile you should never move.

△ Trap: A forwarded target needs the right variables exported

`make app PROJECT=rxchain` works only because `PROJECT`, `SOURCE` and `HEEP_DIR` are exported *before* the `include`: X-HEEP’s own targets read them as environment variables, not as this Makefile’s local variables. Forgetting `export` on a variable a forwarded target needs is the most common way this pattern breaks.

3.5 Reading a Makefile you have not seen before

In order: run `make help` if one exists (both labs define it) for a one-screen summary. Then open the file and read top to bottom once, not target by target – the variable definitions near the top explain every recipe below them. Finally, for the one target you actually want to run, read its prerequisites line before its recipe: half of what a target does in these labs is “make sure this other thing ran first”.

3.6 Adding a target of your own

1. Pick a short, verb-like name (`waves`, not `open-the-waveform-viewer`).
2. Mark it `.PHONY` unless it genuinely produces a file named after the target.
3. List real prerequisites, not commands: if the target needs vectors to exist, depend on the `vectors` target, do not repeat its recipe.
4. Reuse the Makefile’s own variables (`$(FUSESOC)`, `$(CORE)`, `$(ROOT)`) instead of hardcoding paths or tool names – that is what makes `make questa` and `make sim` in Lab 1 nearly identical recipes.
5. Add one line to the `help` target. An undocumented target is, for the next person, an unwritten one.

3.7 How it all fits together: one `make sim run`

Chapters 1–3 each covered one piece in isolation; here is `make sim` in Lab 1 exercising all three at once, top to bottom.

1. `make sim` first checks its prerequisites, `vectors` and `bitexact`: `vectors` runs the Python golden model to produce the expected CORDIC outputs, `bitexact` checks that model against the cookbook’s own reference (Chapter 4 is unrelated to this pair – they never touch the register interface).
2. Only once both succeed does `sim`’s own recipe run: a `fusesoc run -target sim` with `$(FUSESOC_FLAGS)` supplying both `-cores-root` directories from Chapter 1 and `-run_options` passing the vector directory to the testbench as a plusarg.
3. FuseSoC resolves `isa:lab1:cordic_accel`’s dependency tree – `obi`, `register_interface`, `common_cells` and the others named in the `.core` file’s `depend` lists – against whatever it finds under the vendored `vendor/` tree from Chapter 2. A dependency that was never vendored, or was vendored into the wrong `-cores-root`, fails here with “core not found”, not later.
4. With the dependency graph resolved, FuseSoC writes the file list into `build/isa_lab1_cordic_accel_0.1.0` (Section 1.5) and invokes Verilator with the `verilator_options` from the `sim` target – including the `-Wno-fatal` from the `gotcha` in Chapter 1.

5. Verilator compiles and runs the testbench, which reads back the vectors `vectors` generated and prints a pass/fail summary; that exit status is all `make sim` ultimately reports.

Recipe: Where `make regs` fits

`regs` is not a prerequisite of `sim`: it is a separate, manual step you run only after editing `data/cordic_accel.hjson`, to regenerate the register RTL and header *before* the next `make sim`. Chapter 4 covers what it generates and why.

3.8 Checklist

- A Makefile target is a name for a command; read the recipe, not just the name, to know what actually runs.
- `.PHONY` targets are actions, not files; almost every target in these labs is one.
- `export` a variable before an `include` if a forwarded target needs to see it.
- `include <file>` must come last if the included file defines a catch-all rule.
- `make help` first, then read top to bottom, then read the one target you need – prerequisites before recipe.

The Register Interface and Regtool

The CSR half of an accelerator: the `register_interface` bus that carries control/status/configuration, and `regtool` ("reggen"), which turns one `.hjson` description into the RTL and the C header for it. `make regs` is the Makefile target; this chapter is what it actually does.

4.1 Two buses, one job each

`cordic_accel.sv` (Chapter 1) has two interfaces, and they are not interchangeable: an **OBI** port moves the angles and results (the data), and a **register interface** port moves configuration – write SRC, DST, N, write CTRL.START, poll STATUS.DONE. Splitting them is deliberate: the CPU only ever touches a handful of small registers, and the accelerator moves the bulk data itself over OBI, which is where the actual speedup comes from.

`register_interface` (pulp-platform, vendored the same way as Chapter 2 describes) defines that second bus: a request struct (address, write, write data, byte strobes) and a response struct (read data, error), typedef'd for one specific address/data width with the same kind of macro Chapter 1 showed for OBI:

```
typedef logic [31:0] reg_addr_t;
typedef logic [31:0] reg_data_t;
typedef logic [ 3:0] reg_strb_t;

`REG_BUS_TYPEDEF_ALL(cordic_reg, reg_addr_t, reg_data_t, reg_strb_t)
```

Listing 4.1. `rtl/cordic_accel_types_pkg.sv` – the reg bus types

You will not hand-write anything that speaks this protocol directly. You describe your registers in `.hjson`, `regtool` generates a block that speaks `reg_req/reg_rsp` on one side, and your own design only ever touches plain struct fields on the other.

4.2 Anatomy of the `.hjson` register description

```
{ name: "cordic_accel"
  clock_primary: "clk_i"
  reset_primary: "rst_ni"
  bus_interfaces: [
    { protocol: "reg_iface", direction: "device" }
  ]
  regwidth: 32
  registers: [

    { name:      "CTRL"
      desc:      "Control register. Write START=1 to launch a run."
    }
```

```

    swaccess: "wo"
    hwaccess: "hro"
    hwqe:      "true"
    fields: [
        { bits: "0", name: "START"
          desc: "Write 1 to start processing N angles."
        }
    ]
}

{ name:      "STATUS"
  desc:      "Status register, driven by hardware."
  swaccess:  "ro"
  hwaccess:  "hwo"
  hwext:     "true"
  fields: [
    { bits: "0", name: "DONE" desc: "Set when results are ready." }
    { bits: "1", name: "BUSY" desc: "Set while the port is owned." }
  ]
}
]
}

```

Listing 4.2. data/cordic_accel.hjson (trimmed to two registers)

The header names the block and its clock/reset, and `regwidth` fixes every register at the same width (32, here). Offsets are not written down anywhere: `regtool` assigns them in declaration order, word aligned – CTRL at 0x00, STATUS at 0x04, and so on. Each register then has:

- **swaccess**: what *software* may do – wo (write-only), ro (read-only) or rw.
- **hwaccess**: what *hardware* may do – hro (hardware reads the value, never writes it: the normal case for a configuration register like CTRL or N) or hwo (hardware writes it, software only reads: the normal case for a status register). Consult `regtool`'s own documentation for the remaining combinations; these two cover everything in this example.
- **resval**: the reset value, when the register needs one other than zero (Listing 4.2 omits it here, but SEED_X in the full file resets to the CORDIC gain constant).
- **fields**: one or more named bit ranges inside the register, each with its own **bits** and **desc**.

△ **Trap: hwqe and hwext look similar and do opposite jobs**

hwqe: true adds a one-cycle write-strobe your hardware can see (`reg2hw.ctrl.qe`), on top of the stored value (`reg2hw.ctrl.q`). Use it when hardware needs to know *that a write just happened*, not only the value – Section 4.5 shows exactly this, detecting a write of START=1.

hwext: true removes the register's own storage: a software read returns whatever `hw2reg` is driving *this cycle*, combinationally, instead of a value latched on some earlier cycle. STATUS needs this because, without it, software that writes START and immediately reads STATUS would see the flopped value from before the write – the previous run's DONE – and wrongly conclude the new run had already finished.

4.3 What make regs actually runs

```

python3 vendor/pulp_platform/register_interface/vendor/lowrisc_opentitan/util/regtool.py \
-r -t rtl data/cordic_accel.hjson

```

```
python3 ../regtool.py -D -o sw/cordic_accel_regs.h data/cordic_accel.hjson
```

regtool (“reggen”) is OpenTitan’s register-generation tool. It ships vendored inside `register_interface` (Chapter 2 fetched it along with everything else in that IP), so it is present the moment `make vendor` has run; nothing extra to install. Two runs, two outputs:

- `-r -t rtl`: writes `rtl/cordic_accel_reg_pkg.sv` (the struct *types*) and `rtl/cordic_accel_reg_top.sv` (the module that speaks the bus and exposes those structs).
- `-D -o <file>`: writes a C header of offsets, bit positions and masks – one `#define` block per register, generated straight from the same `desc` strings, so the comments in `cordic_accel_regs.h` and in the `.hjson` match by construction.

The lab Makefile wraps both calls as `make regs` (Chapter 3); re-run it whenever `data/cordic_accel.hjson` changes, the same way you re-run `make vectors` after changing the golden model.

△ **Trap:** The generated files are not yours to edit

`cordic_accel_reg_pkg.sv`, `cordic_accel_reg_top.sv` and `cordic_accel_regs.h` all say “auto-generated” at the top. Editing them by hand works until the next `make regs` silently overwrites your change. Edit `data/cordic_accel.hjson` and regenerate instead.

4.4 Reading the generated types

regtool turns every register into a small packed struct, and groups them into one `reg2hw_t` (hardware reads this) and one `hw2reg_t` (hardware drives this):

```
typedef struct packed {
    logic q;    // the stored value
    logic qe;   // one-cycle pulse: software wrote this cycle
} cordic_accel_reg2hw_ctrl_reg_t;

typedef struct packed {
    struct packed { logic d; } done; // hardware drives this bit
    struct packed { logic d; } busy;
} cordic_accel_hw2reg_status_reg_t;
```

Listing 4.3. `rtl/cordic_accel_reg_pkg.sv` (generated, excerpt)

The naming is consistent everywhere this pattern is used, in every X-HEEP peripheral and not only this one: a `reg2hw` field ends in `.q` (the value) and, if `hwqe` was set, `.qe` (the write pulse); a `hw2reg` field ends in `.d` (the value your hardware should drive). Once you can read this struct, you can read the CSR block of any peripheral built the same way.

4.5 Wiring it into your design

`cordic_accel.sv` instantiates the generated `reg_top` once and never touches the bus signals again:

```
cordic_accel_reg2hw_t reg2hw;
cordic_accel_hw2reg_t hw2reg;

cordic_accel_reg_top #(
    .reg_req_t (reg_req_t),
    .reg_rsp_t (reg_rsp_t)
```

```

) u_reg_top (
    .clk_i, .rst_ni,
    .reg_req_i,
    .reg_rsp_o,
    .reg2hw,
    .hw2reg,
    .devmode_i (1'b1)
);

// Software wrote START=1 this cycle -> launch a run.
assign start = reg2hw.ctrl.qe & reg2hw.ctrl.q;
...
// Drive STATUS back to software.
assign hw2reg.status.done.d = done_q & ~run_start;
assign hw2reg.status.busy.d = busy_q | run_start;

```

Listing 4.4. rtl/cordic_accel.sv – instantiating the CSR block

From here on the rest of the module is ordinary SystemVerilog: an FSM reads `reg2hw.n.q`, `reg2hw.src.q`, `reg2hw.dst.q` to know what to fetch and where to write it, and drives `hw2reg.status` combinationally so a read of `STATUS` is never stale. Nothing downstream of `u_reg_top` knows or cares that a register bus exists at all.

4.6 The C side

The generated header gives software plain integer offsets and bitfield helpers, nothing more:

```

#define CORDIC_ACCEL_CTRL_REG_OFFSET 0x0
#define CORDIC_ACCEL_CTRL_START_BIT 0

#define CORDIC_ACCEL_STATUS_REG_OFFSET 0x4
#define CORDIC_ACCEL_STATUS_DONE_BIT 0
#define CORDIC_ACCEL_STATUS_BUSY_BIT 1

```

Listing 4.5. sw/cordic_accel_regs.h (generated, excerpt)

Writing an actual driver around these offsets is a Lab 3 problem (the accelerator has to be mapped into X-HEEP’s address space first); this booklet stops at “where these numbers come from”.

4.7 Checklist

- The register interface carries configuration; OBI (or whichever data bus a design uses) carries data. Do not mix the two.
- One `.hjson` per block; `swaccess/hwaccess` say who may read or write each register, `hwqe` adds a write-pulse, `hwext` removes the register’s own storage.
- make `regs` runs `regtool` twice: once for the RTL (`reg_pkg.sv` + `reg_top.sv`), once for the C header.
- `reg2hw.<reg>.q/.qe` is what hardware reads; `hw2reg.<reg>.d` is what hardware drives. Every peripheral built this way uses the same two suffixes.
- Never hand-edit a generated file; edit the `.hjson` and re-run `make regs`.

One-Page Quick Reference

Every command and file field in this booklet, compressed onto one page to keep next to the keyboard.

A.1 Commands

Command	Purpose
<code>fusesoc -cores-root <dir> ...</code>	Add <code><dir></code> to where FuseSoC looks for <code>.core</code> files. Repeat once per root you need.
<code>fusesoc ... run -target <t> <core></code>	Resolve dependencies and run target <code><t></code> of <code><core></code> .
<code>fusesoc ... run -target <t> <core></code> <code>-run_options="..."</code>	Pass arguments straight through to the simulation binary (plusargs).
<code>python3 util/vendor.py</code> <code><descriptor>.vendor.hjson</code>	Fetch one dependency at its pinned revision into <code>vendor/</code> .
<code>make vendor</code>	Run the vendoring script over every <code>*.vendor.hjson</code> the lab has.
<code>make help</code>	List a lab's own Makefile targets with a one-line description each.
<code>make <target> -dry-run</code>	Print the commands a target would run, without running them.
<code>python3 regtool.py -r -t rtl</code> <code><regs>.hjson</code>	Generate <code>*_reg_pkg.sv</code> and <code>*_reg_top.sv</code> from a register description.
<code>python3 regtool.py -D -o <regs>.h</code> <code><regs>.hjson</code>	Generate the matching C header of offsets, masks and field macros.
<code>make regs</code>	Run both regtool invocations above for a lab that defines this target.

A.2 .core file fields

Field	Meaning
<code>CAP1=2:</code>	Required first line; marks the file as Core API v2.
<code>name</code>	<code>vendor:library:core:version</code> , the string a dependant names in its own <code>depend</code> list.
<code>filesets.<name>.files</code>	Source paths, relative to the <code>.core</code> file, in compile order.
<code>filesets.<name>.file_type</code>	<code>systemVerilogSource</code> , <code>vhdlSource</code> , <code>cSource</code> , ...
<code>filesets.<name>.depend</code>	Other cores' names; FuseSoC resolves them recursively.

Field	Meaning
<code>targets.<name>.filesets</code>	Which filesets this target pulls in.
<code>targets.<name>.default_tool</code>	Which tool block under <code>tools</code> to use.
<code>targets.<name>.toplevel</code>	The module to elaborate from.
<code>targets.<name>.tools.<tool></code>	Tool-specific options (<code>verilator_options</code> , <code>vlog_options</code> , ...).

A.3 .vendor.hjson fields

Field	Meaning
<code>name</code>	A label for the descriptor (not a FuseSoC core name).
<code>target_dir</code>	Where the snapshot is copied to.
<code>upstream.url</code>	The upstream repository.
<code>upstream.rev</code>	The pinned tag, branch or commit – the whole reproducibility guarantee.
<code>exclude_from_upstream</code>	Paths dropped from the copy (tests, CI).

A.4 .hjson register description fields

Field	Meaning
<code>name</code> / <code>regwidth</code>	The block's name (prefixes every generated symbol) and bus width in bits (normally 32).
<code>registers[] .name</code> / <code>.desc</code>	One entry per register; <code>desc</code> ends up as a comment in the generated header.
<code>swaccess</code>	Software access: <code>ro</code> , <code>wo</code> or <code>rw</code> – what the CPU is allowed to do at this address.
<code>hwaccess</code>	Hardware access: <code>hro</code> (hardware reads <code>.q</code> only) or <code>hwo</code> (hardware writes <code>.d</code> only).
<code>hwqe</code>	Adds a one-cycle <code>.qe</code> write-strobe alongside <code>.q</code> , so hardware can see <i>when</i> software wrote, not just the value.
<code>hwext</code>	Removes the register's internal storage; <code>hw2reg</code> must drive the read value combinatorially every cycle.
<code>resval</code>	Reset value, in the generated hardware and in documentation.
<code>fields[] .bits</code> / <code>.name</code>	Sub-fields of a register, each with its own bit range and name (e.g. "0" for DONE).

A.5 Writing a new .core file, in five steps

1. Name it: `<vendor>:<lab>:<core>:<version>`.
2. List your sources in a fileset, in compile order.
3. Declare dependencies by name in `depend`; vendor them first if they are not already under `vendor/`.
4. Add one target per tool you run, starting with `lint`.
5. Set `toplevel` to what that target should elaborate.

A.6 The three traps to remember

1. A `.core` file's filesets do nothing until a target lists them; check the target block first when a build seems to be missing files.
2. `upstream.rev:` `"main"` is fine while developing, wrong once a lab ships: pin it to a SHA or tag.
3. `include` of another Makefile must come last if that file defines a catch-all rule, or every target after it is invisible.
4. Generated files (`*_reg_pkg.sv`, `*_reg_top.sv`, `*_regs.h`) are overwritten by `make regs`; edit the `.hjson` source, never the generated files themselves.